



DL Performance Improvement

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

Deep Learning Neural Network-এর Performance উন্নত করার কৌশলসমূহ

বিষয়: How to Improve Performance of Deep Learning Neural Network

ভাষা: সম্পূর্ণ বাংলা (Technical terms ইংরেজিতে)

স্তর: Intermediate to Advanced

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

এই concept কী এবং কেন দরকার?

কল্পনা করো তুমি একজন ক্রিকেট কোচ। তোমার দলের খেলোয়াড়রা অনুশীলনে ভালো খেলে, কিন্তু আসল ম্যাচে খারাপ করে। তুমি কী করবে?

- **নতুন কৌশল** শেখাবে (নতুন Architecture)
- **বেশি অনুশীলন** করাবে ভালো মাঠে (Data Augmentation)
- **ক্লান্তি হলে বিশ্রাম** দেবে (Early Stopping)
- **অযথা নড়াচড়া** কমাবে (Regularization)
- **সেরা কোচিং পদ্ধতি** ব্যবহার করবে (Optimizer Tuning)

Deep Learning-এও ঠিক একই ব্যাপার। একটি Neural Network তৈরি করলেই কাজ শেষ না — সেটা আসল ডেটায় (test/real world) কতটা ভালো কাজ করে সেটাই মূল প্রশ্ন।

বাস্তব জীবনের উদাহরণ

উদাহরণ ১ — রান্নার রেসিপি:

একটি রেস্টোরাঁর রাঁধুনি প্রথমবার রান্না করলে নানা ভুল করে। সে ধীরে ধীরে:

- উপকরণের পরিমাণ ঠিক করে (Hyperparameter Tuning)
- গ্যাসের আঁচ নিয়ন্ত্রণ করে (Learning Rate)
- অতিরিক্ত লবণ দেওয়া বন্ধ করে (Overfitting রোধ)
- নতুন রেসিপি শেখে পুরনো দক্ষতা ধরে রেখে (Transfer Learning)

উদাহরণ ২ — একটি ছাত্র:

পরীক্ষার আগে একজন ছাত্র:

- পুরনো প্রশ্নপত্র দেখে (Transfer Learning)
- বারবার মুখস্থ না করে বুঝে পড়ে (Regularization)
- ঘুম ও বিশ্রাম নেয় (Early Stopping)
- শুধু গুরুত্বপূর্ণ টপিকে মনোযোগ দেয় (Feature Selection)

এটি কোন সমস্যা সমাধান করে?

সমস্যা	সমাধান কৌশল
Model খুব বেশি ভুল করে (Underfitting)	Architecture বড় করো, বেশি epoch
Training-এ ভালো, Test-এ খারাপ (Overfitting)	Dropout, L2 Regularization
Training খুব ধীরে হয়	Batch Normalization, Learning Rate Scheduling
Gradient হারিয়ে যায় (Vanishing Gradient)	He Initialization, ReLU, Skip Connections
Data কম	Data Augmentation, Transfer Learning

২. বিস্তারিত ব্যাখ্যা (Deep Explanation)

Deep Learning-এ performance উন্নত করার কৌশলগুলোকে ৬টি প্রধান ক্যাটাগরিতে ভাগ করা যায়:

২.১ Data-Level কৌশল

২.১.১ Data Preprocessing (ডেটা প্রস্তুতি)

Raw ডেটা সরাসরি দিলে Neural Network ভালো কাজ করে না। কারণ বিভিন্ন feature-এর scale আলাদা হলে Gradient Descent সঠিকভাবে কাজ করতে পারে না।

Normalization (নরমালাইজেশন):

ডেটাকে [0, 1] বা [-1, 1] এর মধ্যে নিয়ে আসা।

Min-Max Normalization:

$$X_{\text{normalized}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

Standardization (স্ট্যান্ডার্ডাইজেশন):

ডেটার Mean বাদ দিয়ে Standard Deviation দিয়ে ভাগ করা।

$$\text{Z-score: } X_{\text{standardized}} = (X - \mu) / \sigma$$

২.১.২ Data Augmentation (ডেটা বৃদ্ধি করা)

কম ডেটা থেকে বেশি ডেটা তৈরি করার কৌশল। যেমন একটি বিড়ালের ছবি থেকে:

- **Rotation** — ছবি ঘোরানো (15°, 30°, 45°)
- **Flipping** — বাম-ডান উল্টানো
- **Zoom** — ছবি zoom in/out করা
- **Brightness Change** — আলো কম-বেশি করা
- **Noise Addition** — হালকা noise যোগ করা

এই কৌশলে ১০,০০০ ছবি থেকে ১,০০,০০০ ছবির সমতুল্য ডেটা তৈরি করা যায়।

২.১.৩ Handling Class Imbalance (অসম ডেটা সামলানো)

যদি ৯৫% ডেটা "normal" এবং ৫% ডেটা "cancer" হয়, তাহলে model সবকিছুকে "normal" বলে ভালো accuracy পাবে কিন্তু useful হবে না।

সমাধান:

- **Oversampling** — কম ডেটার class থেকে বেশি নমুনা নেওয়া (SMOTE technique)
- **Undersampling** — বেশি ডেটার class কমিয়ে আনা
- **Class Weights** — কম ডেটার class-কে বেশি weight দেওয়া

২.২ Architecture-Level কৌশল

২.২.১ Weight Initialization (ওজন প্রাথমিকীকরণ)

Neural Network শুরুতে কী weight নেবে সেটা অত্যন্ত গুরুত্বপূর্ণ।

সমস্যা:

- সব weight = 0 দিলে → সব neuron একই শিখবে (Symmetry Problem)
- অনেক বড় weight → Exploding Gradient
- অনেক ছোট weight → Vanishing Gradient

সমাধান:

Initialization	Activation Function	সূত্র
Xavier/Glorot	Sigmoid, Tanh	$W \sim N(0, 2/(n_{in} + n_{out}))$
He Initialization	ReLU, Leaky ReLU	$W \sim N(0, 2/n_{in})$
LeCun	SELU	$W \sim N(0, 1/n_{in})$

২.২.২ Batch Normalization (ব্যাচ নরমালাইজেশন)

Deep Network-এ প্রতিটি layer-এর input-এর distribution পরিবর্তন হতে থাকে। এটাকে বলে **Internal Covariate Shift**।

কীভাবে কাজ করে:

1. একটি mini-batch-এর সব activation-এর **Mean** ও **Variance** বের করো
2. সেগুলোকে **Normalize** করো (mean=0, variance=1)
3. তারপর শিক্ষণীয় parameter γ ও β দিয়ে **Scale** ও **Shift** করো

ধাপ ১: $\mu_B = (1/m) \sum x_i$ (batch mean)

ধাপ ২: $\sigma^2_B = (1/m) \sum (x_i - \mu_B)^2$ (batch variance)

ধাপ ৩: $\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)}$ (normalize)

ধাপ ৪: $y_i = \gamma \cdot \hat{x}_i + \beta$ (scale and shift)

সুবিধা:

- দ্রুত training

- Higher learning rate ব্যবহার করা যায়
- Dropout-এর প্রয়োজনীয়তা কমে

২.২.৩ Skip Connections / Residual Connections

ResNet-এ ব্যবহৃত এই কৌশলে layer-এর output-এর সাথে input-কে সরাসরি যোগ করা হয়।

সাধারণ: $\text{output} = F(x)$

Residual: $\text{output} = F(x) + x$

এতে:

- Vanishing Gradient সমস্যা কমে
- ১০০০+ layer পর্যন্ত Network তৈরি করা সম্ভব হয়

২.৩ Regularization কৌশল (Overfitting রোধ)

Overfitting মানে model training data-তে অনেক ভালো কিন্তু নতুন data-তে খারাপ করে।

২.৩.১ L1 এবং L2 Regularization

Loss function-এ weight-এর penalty যোগ করা হয়।

L2 (Ridge/Weight Decay) — সবচেয়ে জনপ্রিয়:

$$\text{Loss}_{\text{total}} = \text{Loss}_{\text{original}} + \lambda \cdot \sum w^2$$

L1 (Lasso):

$$\text{Loss}_{\text{total}} = \text{Loss}_{\text{original}} + \lambda \cdot \sum |w|$$

λ (lambda) = Regularization strength (hyperparameter)

২.৩.২ Dropout

Training-এর সময় প্রতিটি step-এ কিছু neuron randomly "বন্ধ" করে দেওয়া হয়।

Dropout rate $p = 0.5$ মানে:

প্রতিটি neuron 50% সম্ভাবনায় বন্ধ হবে।

গুরুত্বপূর্ণ:

- Training-এর সময়: Dropout **চালু**
- Testing/Inference-এর সময়: Dropout **বন্ধ**
- সব neuron সক্রিয় থাকে কিন্তু weight $(1-p)$ দিয়ে scale করা হয়

২.৩.৩ Early Stopping

Validation loss যখন বাড়তে শুরু করে, training বন্ধ করে দেওয়া।

Epoch	Train Loss	Val Loss	Action
-----	-----	-----	-----
10	0.45	0.50	Continue
20	0.30	0.38	Continue
30	0.15	0.35	Best! Save model
40	0.08	0.42	Overfitting শুরু
50	0.04	0.55	Stop! (patience expired)

Patience = কতটা epoch অপেক্ষা করবো improvement ছাড়া

২.৪ Optimization কৌশল

২.৪.১ Learning Rate Scheduling

Learning rate ধীরে ধীরে কমিয়ে আনলে model ভালো convergence করে।

ধরনসমূহ:

- **Step Decay** — প্রতি N epoch-এ LR অর্ধেক করে
- **Cosine Annealing** — cosine curve অনুসরণ করে LR কমাও
- **ReduceLROnPlateau** — Validation loss না কমলে LR কমাও
- **Warmup + Decay** — শুরুতে ছোট LR, তারপর বড়, তারপর আবার ছোট

২.৪.২ Gradient Clipping

Exploding Gradient রোধ করতে gradient-এর সর্বোচ্চ মান নির্ধারণ করা।

```
if ||gradient|| > threshold:  
    gradient = gradient * (threshold / ||gradient||)
```

২.৪.৩ Mixed Precision Training

FP32 (32-bit) এর বদলে **FP16** (16-bit) ব্যবহার করে:

- মেমরি ব্যবহার অর্ধেক হয়
- Training ২-৩ গুণ দ্রুত হয়
- GPU-তে Tensor Core ব্যবহার করা যায়

২.৫ Transfer Learning (ট্রান্সফার লার্নিং)

লক্ষ লক্ষ ছবি দিয়ে pre-trained একটি model (যেমন ResNet, VGG, BERT) নিয়ে তোমার নিজের কাজে লাগানো।

কৌশল:

Phase 1: Pre-trained model-এর সব layer freeze করো
→ শুধু নতুন output layer train করো (high LR)

Phase 2: কিছু layer unfreeze করো
→ পুরো model fine-tune করো (very low LR)

কখন ব্যবহার করবে:

- তোমার কাছে কম ডেটা আছে
- Pre-trained domain তোমার কাজের সাথে মিলে (যেমন ImageNet → Medical Images)

২.৬ Hyperparameter Tuning

Model-এর বাইরের parameter যেগুলো training-এর আগে সেট করতে হয়:

Hyperparameter	সাধারণ range	প্রভাব
Learning Rate	0.001 - 0.0001	Convergence speed
Batch Size	16 - 512	Memory vs. Speed
Epochs	10 - 1000	Training duration
Dropout Rate	0.2 - 0.5	Regularization strength
L2 λ	0.001 - 0.0001	Weight penalty
Hidden Units	64 - 1024	Model capacity

Search Strategy:

- **Grid Search** — সব combination চেষ্টা (ধীর, exhaustive)
- **Random Search** — Random combination (দ্রুত, practical)
- **Bayesian Optimization** — পূর্ববর্তী result থেকে শিক্ষা নিয়ে search (সবচেয়ে দক্ষ)

৩. Math / Theory

৩.১ L2 Regularization (Math)

সাধারণ Loss:

$$L = (1/n) \sum (y_i - \hat{y}_i)^2 \quad \text{(Mean Squared Error)}$$

L2 যোগ করলে:

$$L_{\text{reg}} = L + \lambda \cdot (1/n) \cdot \sum_j w_j^2$$

যেখানে:

L = Original Loss (MSE, Cross-Entropy ইত্যাদি)

λ = Regularization parameter (0 = no reg, বড় = বেশি penalty)

w_j = j -তম layer-এর weight

n = sample সংখ্যা

Weight Update (Gradient Descent with L2):

$$w_{\text{new}} = w_{\text{old}} - \alpha \cdot (\partial L / \partial w + \lambda \cdot w_{\text{old}})$$

$$w_{\text{new}} = (1 - \alpha \cdot \lambda) \cdot w_{\text{old}} - \alpha \cdot \partial L / \partial w$$

"Weight Decay" – প্রতিটি step-এ weight একটু কমে আসে

সহজ হিসাব:

ধরি: $w = 0.8$, $\alpha = 0.01$, $\lambda = 0.1$, $\partial L / \partial w = 0.3$

$$w_{\text{new}} = (1 - 0.01 \times 0.1) \times 0.8 - 0.01 \times 0.3$$

$$= (1 - 0.001) \times 0.8 - 0.003$$

$$= 0.999 \times 0.8 - 0.003$$

$$= 0.7992 - 0.003$$

$$= 0.7962$$

৩.২ Batch Normalization (Math)

একটি mini-batch $B = \{x_1, x_2, \dots, x_m\}$

ধাপ ১ – Batch Mean:

$$\mu_B = (1/m) \cdot \sum_i x_i$$

ধাপ ২ – Batch Variance:

$$\sigma^2_B = (1/m) \cdot \sum_i (x_i - \mu_B)^2$$

ধাপ ৩ – Normalize:

$$\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)}$$

($\epsilon = 1e-8$, শূন্য দিয়ে ভাগ এড়াতে)

ধাপ ৪ – Scale and Shift:

$$y_i = \gamma \cdot \hat{x}_i + \beta$$

($\gamma, \beta = \text{learnable parameters}$)

সহজ হিসাব:

Batch: $x = [2, 4, 6, 8]$

$$\mu_B = (2+4+6+8)/4 = 5$$

$$\begin{aligned} \sigma^2_B &= [(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2] / 4 \\ &= [9 + 1 + 1 + 9] / 4 = 5 \end{aligned}$$

$$\begin{aligned} \hat{x} &= [(2-5)/\sqrt{5}, (4-5)/\sqrt{5}, (6-5)/\sqrt{5}, (8-5)/\sqrt{5}] \\ &= [-1.34, -0.45, 0.45, 1.34] \end{aligned}$$

$$(\gamma=1, \beta=0 \text{ ধরলে}) y = \hat{x} = [-1.34, -0.45, 0.45, 1.34]$$

৩.৩ He Initialization (Math)

ReLU activation-এর জন্য:

$W \sim N(0, \sigma^2)$ যেখানে $\sigma^2 = 2 / n_{in}$

n_{in} = পূর্ববর্তী layer-এর neuron সংখ্যা

উদাহরণ: পূর্ববর্তী layer-এ 512 neuron থাকলে:

$\sigma = \sqrt{2/512} = \sqrt{0.00390625} \approx 0.0625$

weight গুলো হবে: $N(0, 0.0625^2)$

৩.৪ Learning Rate Decay (Cosine Annealing)

$$LR(t) = LR_{min} + (LR_{max} - LR_{min}) \times (1 + \cos(\pi \times t/T)) / 2$$

যেখানে:

t = বর্তমান epoch

T = মোট epoch সংখ্যা

LR_{max} = সর্বোচ্চ learning rate (শুরু LR)

LR_{min} = সর্বনিম্ন learning rate (সাধারণত 0)

উদাহরণ ($T=100$ epoch, $LR_{max}=0.01$, $LR_{min}=0$):

$t=0$: $LR = 0.01 \times (1 + \cos(0)) / 2 = 0.01$

$t=25$: $LR = 0.01 \times (1 + \cos(\pi/4)) / 2 \approx 0.0085$

$t=50$: $LR = 0.01 \times (1 + \cos(\pi/2)) / 2 = 0.005$

$t=100$: $LR = 0.01 \times (1 + \cos(\pi)) / 2 = 0$

8. Code Example (Python)

```
# =====
# Deep Learning Performance Improvement – Complete Example
# TensorFlow/Keras ব্যবহার করে CIFAR-10 Classification
# =====

import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks
from tensorflow.keras.datasets import cifar10
from sklearn.preprocessing import LabelBinarizer
import matplotlib.pyplot as plt

# — ১. ডেটা লোড এবং Preprocessing —————
print("📁 ডেটা লোড করা হচ্ছে...")
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

# Normalization: pixel values [0, 255] → [0, 1]
X_train = X_train.astype('float32') / 255.0 # training data normalize
X_test = X_test.astype('float32') / 255.0 # test data normalize

# Labels: integer → one-hot encoding
y_train = keras.utils.to_categorical(y_train, 10) # 10 class
y_test = keras.utils.to_categorical(y_test, 10)

print(f" Train data shape: {X_train.shape}") # (50000, 32, 32, 3)
print(f" Test data shape: {X_test.shape}") # (10000, 32, 32, 3)

# — ২. Data Augmentation —————
print("\n🔄 Data Augmentation তৈরি করা হচ্ছে...")
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"), # বাম-ডান উল্টানো
    layers.RandomRotation(0.1), # ±10% ঘোরানো
    layers.RandomZoom(0.1), # ±10% zoom
    layers.RandomTranslation(0.1, 0.1), # ±10% shift
], name="data_augmentation")

# — ৩. Model Architecture (CNN + Batch Norm + Dropout) —
def build_model():
    """
```

Performance-optimized CNN তৈরি করো:

- He Initialization (ReLU-এর জন্য)
 - Batch Normalization (দ্রুত training)
 - Dropout (overfitting রোধ)
 - L2 Regularization (weight penalty)
- """

```

he_init = keras.initializers.HeNormal() # He initialization
l2_reg = keras.regularizers.l2(1e-4) # L2 regularization  $\lambda=0.0001$ 

inputs = keras.Input(shape=(32, 32, 3))

# — Data Augmentation Block (only during training) —
x = data_augmentation(inputs)

# — Convolutional Block 1 —
x = layers.Conv2D(
    32, (3, 3), padding='same',
    kernel_initializer=he_init, # He init ব্যবহার
    kernel_regularizer=l2_reg # L2 regularization
)(x)
x = layers.BatchNormalization()(x) # Batch Normalization
x = layers.Activation('relu')(x) # Activation পরে
x = layers.Conv2D(32, (3, 3), padding='same',
    kernel_initializer=he_init,
    kernel_regularizer=l2_reg)(x)
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)
x = layers.MaxPooling2D(2, 2)(x) # Feature map ছোট করো
x = layers.Dropout(0.25)(x) # 25% neuron randomly বন্ধ

# — Convolutional Block 2 —
x = layers.Conv2D(64, (3, 3), padding='same',
    kernel_initializer=he_init,
    kernel_regularizer=l2_reg)(x)
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)
x = layers.Conv2D(64, (3, 3), padding='same',
    kernel_initializer=he_init,
    kernel_regularizer=l2_reg)(x)
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)
x = layers.MaxPooling2D(2, 2)(x)

```

```

x = layers.Dropout(0.25)(x)          # 25% dropout

# — Convolutional Block 3 —
x = layers.Conv2D(128, (3, 3), padding='same',
                  kernel_initializer=he_init,
                  kernel_regularizer=l2_reg)(x)
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)
x = layers.MaxPooling2D(2, 2)(x)
x = layers.Dropout(0.25)(x)

# — Fully Connected Layers —
x = layers.Flatten()(x)              # 3D → 1D
x = layers.Dense(
    256,
    kernel_initializer=he_init,
    kernel_regularizer=l2_reg
)(x)
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)
x = layers.Dropout(0.5)(x)          # 50% dropout (FC layer-এ বেশি)

outputs = layers.Dense(10, activation='softmax')(x) # Output: 10 class

return keras.Model(inputs, outputs, name="optimized_cnn")

model = build_model()
model.summary() # Model architecture দেখো

# — 8. Learning Rate Scheduling —————
# Cosine Annealing: LR ধীরে ধীরে কমবে
initial_lr = 0.001 # শুরুর Learning Rate

# CosineDecayRestarts: LR কমে তারপর আবার বাড়ে (Warm Restarts)
lr_schedule = keras.optimizers.schedules.CosineDecayRestarts(
    initial_learning_rate=initial_lr,
    first_decay_steps=10,      # প্রথম ১০ epoch-এ একটি cycle
    t_mul=2.0,                 # পরের cycle দ্বিগুণ দীর্ঘ
    m_mul=0.9,                 # প্রতি restart-এ LR একটু কমবে
    alpha=1e-6                 # সর্বনিম্ন LR
)

```

```

# — ৫. Compile (Optimizer + Loss + Metrics) —————
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=lr_schedule),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# — ৬. Callbacks (Early Stopping + Model Checkpoint) ———
callback_list = [
    # Early Stopping: val_loss না কমলে training বন্ধ করো
    callbacks.EarlyStopping(
        monitor='val_loss',    # কী দেখবে
        patience=15,           # ১৫ epoch অপেক্ষা করবে
        restore_best_weights=True, # সেরা weight ফিরিয়ে আনবে
        verbose=1
    ),

    # Model Checkpoint: সেরা model সেভ করো
    callbacks.ModelCheckpoint(
        filepath='best_model.keras', # সেভ পাথ
        monitor='val_accuracy',       # accuracy দেখে সেভ
        save_best_only=True,          # শুধু সেরাটি সেভ
        verbose=1
    ),

    # ReduceLROnPlateau: val_loss না কমলে LR কমাও
    callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,             # LR অর্ধেক করো
        patience=5,             # ৫ epoch অপেক্ষা
        min_lr=1e-7,           # সর্বনিম্ন LR
        verbose=1
    )
]

# — ৭. Training —————
print("\n🚀 Training শুরু হচ্ছে...")
history = model.fit(
    X_train, y_train,
    epochs=100,                # সর্বোচ্চ ১০০ epoch (Early Stopping আগে বন্ধ করবে)

```

```

    batch_size=128,          # একসাথে ১২৮ sample
    validation_split=0.2,    # ২০% validation data
    callbacks=callback_list,
    verbose=1
)

# — ৮. Evaluation —————
print("\n📊 Test Set Evaluation:")
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"  Test Accuracy: {test_acc:.4f} ({test_acc*100:.2f}%)")
print(f"  Test Loss:      {test_loss:.4f}")

# — ৯. Training History Plot —————
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# Accuracy plot
ax1.plot(history.history['accuracy'], label='Train Accuracy')
ax1.plot(history.history['val_accuracy'], label='Val Accuracy')
ax1.set_title('Model Accuracy')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy')
ax1.legend()

# Loss plot
ax2.plot(history.history['loss'], label='Train Loss')
ax2.plot(history.history['val_loss'], label='Val Loss')
ax2.set_title('Model Loss')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
ax2.legend()

plt.tight_layout()
plt.savefig('training_history.png', dpi=150)
plt.show()
print("📈 Training history chart সেভ হয়েছে!")

```

Expected Output:

 ডেটা লোড করা হচ্ছে...

Train data shape: (50000, 32, 32, 3)

Test data shape: (10000, 32, 32, 3)

 Training শুরু হচ্ছে...

Epoch 1/100

313/313 - 12s - loss: 1.8420 - accuracy: 0.3421 - val_loss: 1.6234 - val_accuracy: 0.4198

... (training চলবে) ...

Epoch 45/100

313/313 - 10s - loss: 0.5219 - accuracy: 0.8234 - val_loss: 0.6012 - val_accuracy: 0.8156

Restoring model weights from the end of the best epoch: 38.

 Test Set Evaluation:

Test Accuracy: 0.8123 (81.23%)

Test Loss: 0.6234

```

# -----
# Transfer Learning Example (ResNet50 + Fine-tuning)
# -----

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.applications import ResNet50
from tensorflow.keras import layers

# কল্পনা: তোমার কাছে মাত্র ১০০০ flowers-এর ছবি আছে → ৫ class

# — Phase 1: Feature Extraction —————
print("🔧 Phase 1: Feature Extraction শুরু হচ্ছে...")

# Pre-trained ResNet50 লোড করো (ImageNet weights)
base_model = ResNet50(
    weights='imagenet',      # ১ লক্ষ ছবি দিয়ে pre-trained
    include_top=False,      # শেষের Classification layer বাদ দাও
    input_shape=(224, 224, 3)
)

# Base model freeze করো – এর weight পরিবর্তন হবে না
base_model.trainable = False
print(f"Base model parameters: {base_model.count_params():,}")
print("Base model: Frozen ✓")

# নতুন classification head যোগ করো
x = base_model.output
x = layers.GlobalAveragePooling2D()(x) # Flatten করো
x = layers.Dense(256, activation='relu')(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(5, activation='softmax')(x) # ৫ class flower

model_phase1 = keras.Model(base_model.input, outputs)

# Phase 1 compile (হাই LR)
model_phase1.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3), # High LR
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

```
# Phase 1 train (ধরি X_flowers, y_flowers আছে)
# model_phase1.fit(X_flowers, y_flowers, epochs=10)

# — Phase 2: Fine-tuning —————
print("\n🌀 Phase 2: Fine-tuning শুরু হচ্ছে...")

# Base model-এর শেষ ৩০ layer unfreeze করো
base_model.trainable = True
for layer in base_model.layers[:-30]:
    layer.trainable = False # প্রথম layer গুলো freeze রাখো

trainable_count = sum(1 for l in base_model.layers if l.trainable)
print(f" Trainable layers: {trainable_count}/{len(base_model.layers)}")

# Phase 2 compile (খুব কম LR – catastrophic forgetting এড়াতে)
model_phase1.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-5), # Very Low LR!
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Phase 2 train
# model_phase1.fit(X_flowers, y_flowers, epochs=20)
print(" Fine-tuning ready! (very low LR = 1e-5)")
```

```
# -----  
# Hyperparameter Tuning with Keras Tuner  
# -----  
  
# pip install keras-tuner  
import keras_tuner as kt  
  
def build_tunable_model(hp):  
    """  
    Hyperparameter tuning-এর জন্য model build function।  
    hp object দিয়ে hyperparameter search space define করা হয়।  
    """  
    model = keras.Sequential()  
  
    # Hyperparameter: First Dense layer এর units (64 থেকে 512)  
    units_1 = hp.Int('units_1', min_value=64, max_value=512, step=64)  
    model.add(layers.Dense(units_1, activation='relu'))  
    model.add(layers.BatchNormalization())  
  
    # Hyperparameter: Dropout rate (0.2 থেকে 0.5)  
    dropout_rate = hp.Float('dropout', min_value=0.2, max_value=0.5, step=0.1)  
    model.add(layers.Dropout(dropout_rate))  
  
    # Hyperparameter: Second Dense layer  
    units_2 = hp.Int('units_2', min_value=32, max_value=256, step=32)  
    model.add(layers.Dense(units_2, activation='relu'))  
    model.add(layers.Dropout(dropout_rate))  
  
    model.add(layers.Dense(10, activation='softmax'))  
  
    # Hyperparameter: Learning Rate  
    lr = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])  
  
    model.compile(  
        optimizer=keras.optimizers.Adam(learning_rate=lr),  
        loss='categorical_crossentropy',  
        metrics=['accuracy']  
    )  
    return model  
  
# Bayesian Optimization দিয়ে সেরা hyperparameter খোঁজো  
tuner = kt.BayesianOptimization(  

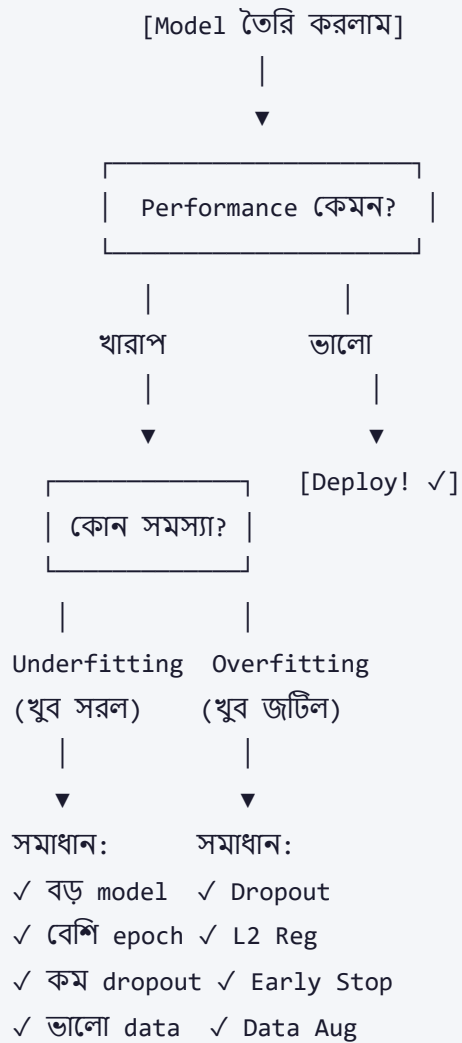
```

```
build_tunable_model,  
objective='val_accuracy', # কোন metric maximize করবো  
max_trials=20,           # কতটি combination চেষ্টা  
directory='hp_search',  
project_name='cifar10_tuning'  
)  
  
# Search শুরু করো  
# tuner.search(X_train, y_train, epochs=30, validation_split=0.2)  
  
# সেরা hyperparameter দেখো  
# best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]  
# print(f"Best units_1: {best_hps.get('units_1')}")  
# print(f"Best dropout: {best_hps.get('dropout')}")  
# print(f"Best LR:      {best_hps.get('learning_rate')}")  
  
print("✅ Hyperparameter Tuning code ready!")
```

🔗 🎨 Visual / Diagram

🔗 Performance Improvement Roadmap

Deep Learning Performance Improvement – Flow Diagram



সাধারণ Performance Improvement Checklist

STEP 1 – DATA

- ✓ Data Normalize/Standardize করো
- ✓ Missing values handle করো
- ✓ Data Augmentation যোগ করো
- ✓ Class imbalance check করো

STEP 2 – ARCHITECTURE

- ✓ He Initialization (ReLU-এর জন্য)
- ✓ Batch Normalization প্রতি layer-এ
- ✓ সঠিক Activation Function
- ✓ Skip Connections (deep network-এ)

|



STEP 3 – REGULARIZATION

- ✓ Dropout (Conv: 0.25, FC: 0.5)
- ✓ L2 Regularization ($\lambda = 1e-4$)
- ✓ Early Stopping (patience=10-20)

|



STEP 4 – OPTIMIZATION

- ✓ Adam Optimizer (LR = 0.001)
- ✓ Learning Rate Scheduling
- ✓ Gradient Clipping (RNN-এ গুরুত্বপূর্ণ)
- ✓ Mixed Precision Training (GPU)

|



STEP 5 – TRANSFER LEARNING

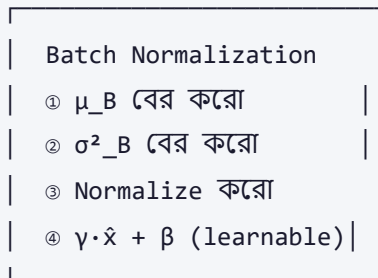
- ✓ Pre-trained model (ResNet, BERT, ViT)
- ✓ Phase 1: Freeze → Train head
- ✓ Phase 2: Unfreeze → Fine-tune (low LR)

৫.২ Overfitting vs Underfitting Diagram



৫.৩ Batch Normalization Architecture

Input Feature Map



Output Feature Map (stable distribution!)

৫.৪ Dropout কীভাবে কাজ করে

Training Time:

Inference Time:

[Input]

```

| | |
o o x ← 33% বন্ধ
| |
x o o ← 33% বন্ধ
| |
o o o
| | |
[Output]

```

[Input]

```

| | |
o o o ← সব চালু
| | |
o o o (×0.67 scale)
| | |
[Output]

```

x = বন্ধ neuron (gradient flow নেই)

o = চালু neuron

৬. Real-world Use Cases

Use Case ১: Google Photos — Image Classification

সমস্যা: বিলিয়ন বিলিয়ন ছবি categorize করতে হবে।

সমাধান:

- **Transfer Learning** — ImageNet-pre-trained EfficientNet
- **Data Augmentation** — বিভিন্ন কোণ, আলো, zoom
- **Batch Normalization** — দ্রুত এবং stable training
- **Distributed Training** — হাজারো GPU-তে একসাথে

ফলাফল: 95%+ accuracy, real-time classification

Use Case ২: Tesla Autopilot — Object Detection

সমস্যা: গাড়ি চালানোর সময় real-time-এ পথচারী, গাড়ি, sign চিনতে হবে।

সমাধান:

- **Mixed Precision Training** — দ্রুত inference

- **Quantization (Model Compression)** — small model for edge device
 - **Ensemble Learning** — একাধিক model একসাথে
 - **Aggressive Data Augmentation** — বৃষ্টি, রাত, ধুলো simulation
-

Use Case ৩: Netflix — Recommendation System

সমস্যা: ২৩ কোটি user-এর জন্য personalized content suggest করা।

সমাধান:

- **Dropout** — Overfitting রোধ (user pattern memorize না করতে)
 - **L2 Regularization** — Generalization বাড়ানো
 - **Learning Rate Scheduling** — Better convergence
 - **Early Stopping** — সঠিক সময়ে থামা
-

Use Case 8: ChatGPT (OpenAI) — Language Model

সমস্যা: Human-quality text generation।

সমাধান:

- **Transfer Learning + Fine-tuning** — GPT-3 → RLHF fine-tuning
 - **Mixed Precision Training** — 175 billion parameter train করা
 - **Gradient Clipping** — Exploding gradient রোধ
 - **Cosine Learning Rate Decay** — Stable convergence
-

Use Case ৫: Radiology AI (Google DeepMind) — Medical Imaging

সমস্যা: X-ray থেকে cancer detect করা।

সমাধান:

- **Transfer Learning** — ImageNet → Medical Images
- **Class Imbalance Handling** — Cancer cases কম, oversampling দরকার
- **Strong Data Augmentation** — Medical image rotation, contrast

- **Ensemble of Models** — Multiple CNN result average করা

ফলাফল: Senior radiologist-এর সমতুল্য accuracy

৭. ⚖️ Pros & Cons

কৌশল	সুবিধা ✓	অসুবিধা ✗
Batch Normalization	দ্রুত training, stable gradient	Inference সময় extra computation, RNN-এ কঠিন
Dropout	Overfitting কমায়, ensemble effect	Training ধীর, inference-এ বন্ধ রাখতে হয়
L2 Regularization	সব weight ছোট রাখে	λ tuning কঠিন
Data Augmentation	কম data থেকে বেশি, generalization	Training time বাড়ে, unrealistic augmentation হতে পারে
Transfer Learning	কম data লাগে, দ্রুত training	Domain mismatch হলে কাজ করে না
Early Stopping	Overfitting রোধ, efficient	Patience wrong হলে premature stopping
Learning Rate Decay	Better convergence	Schedule ভুল হলে diverge করতে পারে
He Initialization	Vanishing gradient কমায়	Activation-specific, wrong use = problem
Mixed Precision	২x দ্রুত, কম memory	Numerical instability হতে পারে

৮. ⚠️ Common Mistakes & Gotchas

ভুল ১: Normalization না করা

```
# ❌ ভুল: raw pixel values দেওয়া
model.fit(X_train, y_train) # X_train value 0-255

# ✅ সঠিক: normalize করো
X_train = X_train / 255.0
model.fit(X_train, y_train)
```

ভুল ২: Dropout Testing-এর সময়ও চালু রাখা

```
# ❌ ভুল: Keras নিজেই handle করে, কিন্তু custom loop-এ ভুল হয়
output = model(x, training=True) # testing-এ True দেওয়া ভুল!

# ✅ সঠিক:
output = model(x, training=False) # inference mode
```

ভুল ৩: খুব বড় Learning Rate দিয়ে শুরু করা

```
# ❌ ভুল – Loss nan হয়ে যায়
optimizer = Adam(learning_rate=0.1) # খুব বড়!

# ✅ সঠিক
optimizer = Adam(learning_rate=0.001) # সাধারণত ভালো শুরু
```

ভুল ৪: Test Data-তে Data Augmentation করা

```
# ❌ ভুল – Test data augment করলে evaluation ভুল হয়
train_aug = ImageDataGenerator(rotation_range=20, horizontal_flip=True)
test_aug = ImageDataGenerator(rotation_range=20) # ← ভুল!

# ✅ সঠিক – Test data শুধু normalize করো
test_aug = ImageDataGenerator(rescale=1./255)
```

ভুল ৫: Transfer Learning-এ সব Layer একসাথে Unfreeze

```
# ❌ ভুল – Catastrophic Forgetting হবে
base_model.trainable = True
model.compile(optimizer=Adam(lr=0.001)) # High LR with all unfrozen!

# ✅ সঠিক – Phase করে unfreeze, low LR
base_model.trainable = True
for layer in base_model.layers[:-30]:
    layer.trainable = False
model.compile(optimizer=Adam(lr=1e-5)) # Very low LR!
```

ভুল ৬: Batch Norm ও Dropout-এর ভুল সিকোয়েন্স

```
# ❌ ভুল ক্রম
x = Dense(256)(x)
x = Dropout(0.5)(x) # Dropout আগে
x = BatchNorm()(x) # BatchNorm পরে – variance unstable!

# ✅ সঠিক ক্রম
x = Dense(256)(x)
x = BatchNorm()(x) # BatchNorm আগে
x = Activation('relu')(x)
x = Dropout(0.5)(x) # Dropout পরে
```

ভুল ৭: Validation Loss না দেখে Train Loss দেখে সিদ্ধান্ত নেওয়া

❌ "Train Loss কমছে → model ভালো হচ্ছে" – ভুল!

✅ সবসময় Train Loss AND Val Loss দুটোই দেখো।
Val Loss বাড়লে = Overfitting (problem!)

ভুল ৮: Learning Rate Scheduler ছাড়া Long Training

```
# ❌ Fixed LR দিয়ে ১০০ epoch  
model.compile(optimizer=Adam(lr=0.001))  
model.fit(epochs=100) # শেষে LR বেশি হয়ে ভালো convergence হয় না  
  
# ✅ LR Scheduler ব্যবহার করো  
callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5)
```

৯. 📌 Related Topics

আগে যা জানা দরকার (Prerequisites)

[মূল ভিত্তি]

- └ Linear Algebra (Matrix, Vector operations)
- └ Calculus (Partial derivatives, Chain Rule)
- └ Probability & Statistics (Mean, Variance, Distribution)
- |

[ML Foundation]

- └ Gradient Descent Algorithm
- └ Loss Functions (MSE, Cross-Entropy)
- └ Overfitting / Underfitting concept
- |

[DL Foundation]

- └ ANN (Artificial Neural Network)
- └ Activation Functions (ReLU, Sigmoid, Softmax)
- └ Backpropagation
- └ Optimizers (SGD, Adam, RMSProp)

পরে কী শেখা উচিত (Next Steps)

[Advanced Regularization]

- └ Label Smoothing
- └ Mixup Augmentation
- └ CutMix
- └ Stochastic Depth

|

[Advanced Architecture]

- └ ResNet (Residual Connections)
- └ EfficientNet (Compound Scaling)
- └ Vision Transformer (ViT)

|

[Advanced Training]

- └ Knowledge Distillation
- └ Pruning & Quantization
- └ Neural Architecture Search (NAS)
- └ Distributed Training (Multi-GPU)

|

[Monitoring Tools]

- └ TensorBoard
- └ Weights & Biases (wandb)
- └ MLflow

So. 🧠 Memory Tricks

মনে রাখার কৌশল: "DRAIN BET"

- D – Data Preprocessing & Augmentation
- R – Regularization (Dropout, L1/L2)
- A – Architecture (BatchNorm, He Init)
- I – Initialization (He, Xavier, LeCun)
- N – Normalization (BatchNorm, LayerNorm)

- B – Batch Size Tuning
- E – Early Stopping
- T – Transfer Learning

ছোট্ট সারণী — "কী সমস্যায় কী করবো"

সমস্যা দেখলে	→ সাথে সাথে মনে করো
Train loss কমছে না	→ LR বাড়াও, He Init চেক করো
Val loss বাড়ছে	→ Dropout বাড়াও, L2 যোগ করো
Training খুব ধীর	→ Batch Norm যোগ করো, LR বাড়াও
Gradient nan হচ্ছে	→ LR কমাও, Gradient Clip করো
কম data আছে	→ Transfer Learning, Data Aug
Model খুব বড়/ধীরে	→ Pruning, Quantization, Distillation

১ লাইনে সারসংক্ষেপ

"ভালো Data দাও → সঠিক Architecture বানাও → Regularization করো → Learning Rate schedule করো → Transfer Learning দিয়ে শুরু করো — এই ৫ ধাপ মানলেই Deep Learning Model-এর Performance ৮০%+ উন্নত হবে।"

সংখ্যা মনে রাখো

Dropout rate: Conv = 0.25, Dense = 0.5
 L2 λ : খুব সাধারণ = $1e-4$
 Learning Rate: Adam default = 0.001 ($1e-3$)
 He Init: $\sigma = \sqrt{2/n_{in}}$
 Batch Norm ϵ : $1e-8$ (শূন্য division এড়াতে)
 Fine-tune LR: pre-trained LR-এর $\sim 10\times$ ছোট

 তৈরির তারিখ: ২০২৬-০৪-০৯

 AI-assisted Bengali ML Notes | সম্পূর্ণ বাংলায়

 Path: D:\Coding\Generative-AI\ML_Notes\DL_Performance_Improvement\

 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

 AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes